

Admin Panel — WebSocket & Real-time Integration

1. Connection

The admin panel connects to the WebSocket server on dashboard mount:

```
ws://{host}/ws/admin/?token={JWT}
```

- Connection is established when the admin dashboard layout mounts.
 - Disconnects on logout (`signOut()`) or browser beforeunload.
 - JWT is retrieved from the next-auth session.
-

2. Events Received from Backend

TypeScript Message Types

```
// types/websocket.ts
```

```
interface WSMMessage<T = unknown> {  
  type: string;  
  payload: T;  
}
```

```
// SOS Events
```

```
interface SOSNewPayload {  
  id: string;  
  user: { id: string; name: string; photo_url?: string };  
  guard: { id: string; name: string; photo_url?: string };  
  booking_id: string;  
  location: { lat: number; lng: number };  
  triggered_at: string;  
}
```

```
interface SOSResolvedPayload {  
  id: string;  
  resolved_by: string;  
  resolved_at: string;  
}
```

```

// Booking Events
interface BookingNewPayload {
    id: string;
    user_id: string;
    status: string;
}

interface BookingCompletedPayload {
    id: string;
}

interface BookingCancelledPayload {
    id: string;
}

// Guard Events
interface GuardOnlinePayload {
    guard_id: string;
}

interface GuardOfflinePayload {
    guard_id: string;
}

// Payment Events
interface PaymentNewTransactionPayload {
    amount: number;
    type: "credit" | "debit";
}

type AdminWSMessage =
    | WSMMessage<SOSNewPayload> & { type: "sos.new" }
    | WSMMessage<SOSResolvedPayload> & { type: "sos.resolved" }
    | WSMMessage<BookingNewPayload> & { type: "booking.new" }
    | WSMMessage<BookingCompletedPayload> & { type:
        "booking.completed" }
    | WSMMessage<BookingCancelledPayload> & { type:
        "booking.cancelled" }
    | WSMMessage<GuardOnlinePayload> & { type: "guard.online" }
    | WSMMessage<GuardOfflinePayload> & { type: "guard.offline" }
    | WSMMessage<PaymentNewTransactionPayload> & { type:
        "payment.new_transaction" };

```

Event Handling Summary

Event	Action
sos.new	Add to active SOS list, play alarm, show browser notification, increment sidebar badge
sos.resolved	

Event	Action
	Remove from active list, stop alarm if no active SOS remain
booking.new	Increment active bookings KPI
booking.completed	Decrement active bookings, increment completed count
booking.cancelled	Decrement active bookings
guard.online	Increment online guards KPI
guard.offline	Decrement online guards KPI
payment.new_transaction	Update revenue today if type === "credit"

3. useAdminWebSocket Hook

```
// hooks/use-admin-websocket.ts
"use client";

import { useEffect, useRef, useState, useCallback } from "react";
import { useSession } from "next-auth/react";
import { SOSNewPayload, AdminWSMessage } from "@types/websocket";

interface AdminMetrics {
  activeBookings: number;
  onlineGuards: number;
  revenueToday: number;
  completedBookings: number;
}

interface UseAdminWebSocketReturn {
  isConnected: boolean;
  connectionState: "connected" | "disconnected" | "reconnecting";
  sosEvents: SOSNewPayload[];
  metrics: AdminMetrics;
  removeSOS: (id: string) => void;
}

export function useAdminWebSocket(
  initialMetrics?: Partial<AdminMetrics>
): UseAdminWebSocketReturn {
  const { data: session } = useSession();
  const wsRef = useRef<WebSocket | null>(null);
  const reconnectAttemptRef = useRef(0);
  const reconnectTimeoutRef = useRef<NodeJS.Timeout>();

  const [connectionState, setConnectionState] = useState<
    "connected" | "disconnected" | "reconnecting"
```

```

>("disconnected");
const [sosEvents, setSOSEvents] = useState<SOSNewPayload[]>([]);
const [metrics, setMetrics] = useState<AdminMetrics>({
  activeBookings: initialMetrics?.activeBookings ?? 0,
  onlineGuards: initialMetrics?.onlineGuards ?? 0,
  revenueToday: initialMetrics?.revenueToday ?? 0,
  completedBookings: initialMetrics?.completedBookings ?? 0,
});

const removeSOS = useCallback((id: string) => {
  setSOSEvents((prev) => prev.filter((e) => e.id !== id));
}, []);

const getReconnectDelay = useCallback(() => {
  const delay = Math.min(1000 * Math.pow(2,
    reconnectAttemptRef.current), 30000);
  return delay;
}, []);

const connect = useCallback(() => {
  if (!session?.accessToken) return;

  const host = process.env.NEXT_PUBLIC_WS_HOST ||
    window.location.host;
  const protocol = window.location.protocol === "https:" ?
    "wss" : "ws";
  const url = `${protocol}://${host}/ws/admin/?token=${
    session.accessToken}`;

  const ws = new WebSocket(url);
  wsRef.current = ws;

  ws.onopen = () => {
    setConnectionState("connected");
    reconnectAttemptRef.current = 0;
  };

  ws.onmessage = (event) => {
    try {
      const message: AdminWSMessage = JSON.parse(event.data);
      handleMessage(message);
    } catch (e) {
      console.error("Failed to parse WS message:", e);
    }
  };
};

ws.onclose = () => {
  setConnectionState("reconnecting");
  const delay = getReconnectDelay();
  reconnectAttemptRef.current += 1;
  reconnectTimeoutRef.current = setTimeout(connect, delay);
};

```

```

ws.onerror = (error) => {
  console.error("WebSocket error:", error);
  ws.close();
};
}, [session?.accessToken, getReconnectDelay]);

const handleMessage = (message: AdminWSMessage) => {
  switch (message.type) {
    case "sos.new":
      setSOSEvents((prev) => [message.payload, ...prev]);
      AlarmService.getInstance().play();
      showSOSNotification(message.payload);
      break;

    case "sos.resolved":
      setSOSEvents((prev) => {
        const updated = prev.filter((e) => e.id !==
          message.payload.id);
        if (updated.length === 0)
          AlarmService.getInstance().stop();
        return updated;
      });
      break;

    case "booking.new":
      setMetrics((prev) => ({ ...prev, activeBookings:
        prev.activeBookings + 1 }));
      break;

    case "booking.completed":
      setMetrics((prev) => ({
        ...prev,
        activeBookings: Math.max(0, prev.activeBookings - 1),
        completedBookings: prev.completedBookings + 1,
      }));
      break;

    case "booking.cancelled":
      setMetrics((prev) => ({
        ...prev,
        activeBookings: Math.max(0, prev.activeBookings - 1),
      }));
      break;

    case "guard.online":
      setMetrics((prev) => ({ ...prev, onlineGuards:
        prev.onlineGuards + 1 }));
      break;

    case "guard.offline":
      setMetrics((prev) => ({
        ...prev,
        onlineGuards: Math.max(0, prev.onlineGuards - 1),

```

```

    ));
    break;

    case "payment.new_transaction":
        if (message.payload.type === "credit") {
            setMetrics((prev) => ({
                ...prev,
                revenueToday: prev.revenueToday +
                    message.payload.amount,
            }));
        }
        break;
    }
};

useEffect(() => {
    connect();
    return () => {
        wsRef.current?.close();
        if (reconnectTimeoutRef.current)
            clearTimeout(reconnectTimeoutRef.current);
    };
}, [connect]);

return {
    isConnected: connectionState === "connected",
    connectionState,
    sosEvents,
    metrics,
    removeSOS,
};
}

function showSOSNotification(payload: SOSNewPayload) {
    if (Notification.permission === "granted") {
        const notification = new Notification("SOS Alert!", {
            body: `${payload.user.name} triggered SOS at $
                {payload.location.lat.toFixed(4)}, $
                {payload.location.lng.toFixed(4)}`,
            icon: "/icons/sos-alert.png",
            tag: `sos-${payload.id}`,
            requireInteraction: true,
        });

        notification.onclick = () => {
            window.focus();
            document.getElementById("sos-
                section")?.scrollIntoView({ behavior: "smooth" });
            notification.close();
        };
    }
}

```

4. Alarm Sound — AlarmService

```
// services/alarm.ts
```

```
export class AlarmService {
  private static instance: AlarmService;
  private audio: HTMLAudioElement | null = null;
  private isMuted = false;

  private constructor() {
    if (typeof window !== "undefined") {
      this.audio = new Audio("/sounds/alarm.mp3");
      this.audio.loop = true;
      this.audio.preload = "auto";
    }
  }

  static getInstance(): AlarmService {
    if (!AlarmService.instance) {
      AlarmService.instance = new AlarmService();
    }
    return AlarmService.instance;
  }

  play() {
    if (this.audio && !this.isMuted) {
      this.audio.currentTime = 0;
      this.audio.play().catch((err) => {
        console.warn("Audio playback failed (user interaction required):", err);
      });
    }
  }

  stop() {
    if (this.audio) {
      this.audio.pause();
      this.audio.currentTime = 0;
    }
  }

  mute() {
    this.isMuted = true;
    this.stop();
  }

  unmute() {
    this.isMuted = false;
  }

  get muted() {
    return this.isMuted;
  }
}
```

```
}  
}
```

5. Browser Notifications

```
// lib/notifications.ts
```

```
export function requestNotificationPermission() {  
  if (typeof window === "undefined") return;  
  if (!("Notification" in window)) return;  
  
  if (Notification.permission === "default") {  
    Notification.requestPermission();  
  }  
}
```

Usage in dashboard layout:

```
// app/admin/layout.tsx  
"use client";  
  
import { useEffect } from "react";  
import { requestNotificationPermission } from "@lib/notifications";  
  
export default function AdminLayout({ children }: { children: React.ReactNode }) {  
  useEffect(() => {  
    requestNotificationPermission();  
  }, []);  
  
  return <>{children}</>;  
}
```

6. Connection Status Indicator

```
// components/layout/connection-status.tsx  
"use client";  
  
import { cn } from "@lib/utils";  
  
interface ConnectionStatusProps {  
  state: "connected" | "disconnected" | "reconnecting";  
}  
  
export function ConnectionStatus({ state }: ConnectionStatusProps)  
{  
  const config = {
```



```

    connected: { color: "bg-green-500", label: "Live" },
    disconnected: { color: "bg-red-500", label: "Disconnected" },
    reconnecting: { color: "bg-orange-500", label:
"Reconnecting..." },
  };

  const { color, label } = config[state];

  return (
    <div className="flex items-center gap-2">
      <div className={cn("h-2 w-2 rounded-full", color, state ===
"reconnecting" && "animate-pulse")} />
      <span className="text-xs text-muted-foreground">{label}</
span>
    </div>
  );
}

```

Mute Button in Topbar

```

// components/layout/alarm-toggle.tsx
"use client";

import { useState } from "react";
import { Button } from "@components/ui/button";
import { Volume2, VolumeX } from "lucide-react";
import { AlarmService } from "@services/alarm";

export function AlarmToggle() {
  const [muted, setMuted] = useState(false);

  const toggle = () => {
    const alarm = AlarmService.getInstance();
    if (muted) {
      alarm.unmute();
    } else {
      alarm.mute();
    }
    setMuted(!muted);
  };

  return (
    <Button variant="ghost" size="icon" onClick={toggle}
title={muted ? "Unmute alarm" : "Mute alarm"}>
      {muted ? <VolumeX className="h-4 w-4" /> : <Volume2
className="h-4 w-4" />}
    </Button>
  );
}

```